LAPORAN PRATIKUM IV & V

TEKNOLOGI IOT II



Oleh :

NAMA : Crosella Rahma Dewi
NPM : 25782093
KELAS : TRI - 3D

Dosen Pengampu :

Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

- **Kode**



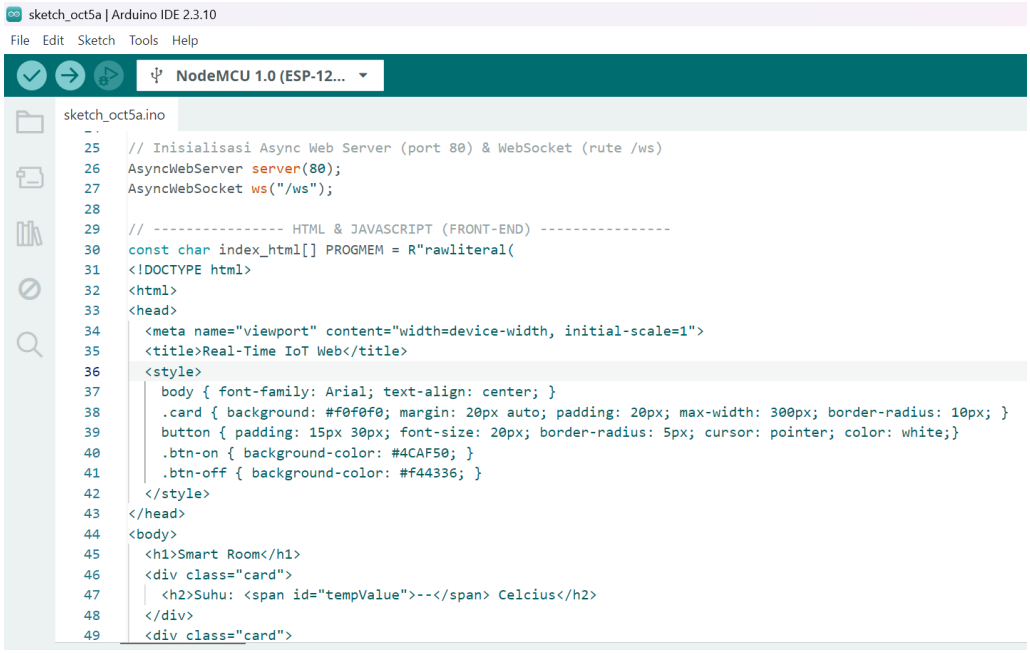

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "kar";
const char* password = "12345678";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;
```




```
// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>

  <script>
    // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    // Menjalankan inisiasi saat halaman pertama kali dimuat
    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
      websocket.onopen    = onOpen;
      websocket.onclose   = onClose;
      websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }
```

```
    // Dipanggil saat tombol di layar web ditekan
    function toggleLed(){
      websocket.send('toggle');
    }

    // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
    function onMessage(event) {
      var dataObj = JSON.parse(event.data);

      // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
      if(dataObj.suhu !== undefined) {
        document.getElementById('tempValue').innerHTML = dataObj.suhu;
      }

      // Merombak UI Tombol & Teks sesuai status hardware
      if(dataObj.led !== undefined) {
        var btn = document.getElementById('toggleBtn');
        var status = document.getElementById('ledStatus');
        if(dataObj.led == "1"){
          status.innerHTML = "ON";
          btn.innerHTML = "Turn OFF";
          btn.className = "btn-on";
        } else {
          status.innerHTML = "OFF";
          btn.innerHTML = "Turn ON";
```

sketch_oct5a.ino

```
          btn.className = "btn-off";
        }
      }
    }
  </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\"}";
  ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
    data[len] = 0;
    if (strcmp((char*)data, "toggle") == 0) {
      ledState = !ledState;
```

```
      notifyClients();
    }
  }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients(); // Kirim status terkini kepada klien yang baru buka web
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);
```

```
void setup() {
  Serial.begin(115200);

  // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
  pinMode(buttonPin, INPUT);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

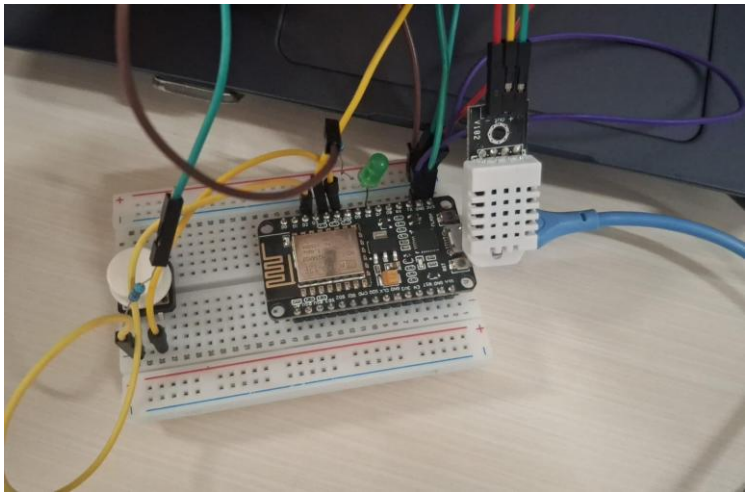
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  // Ikat handler WebSockets ke server utama
  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}
```

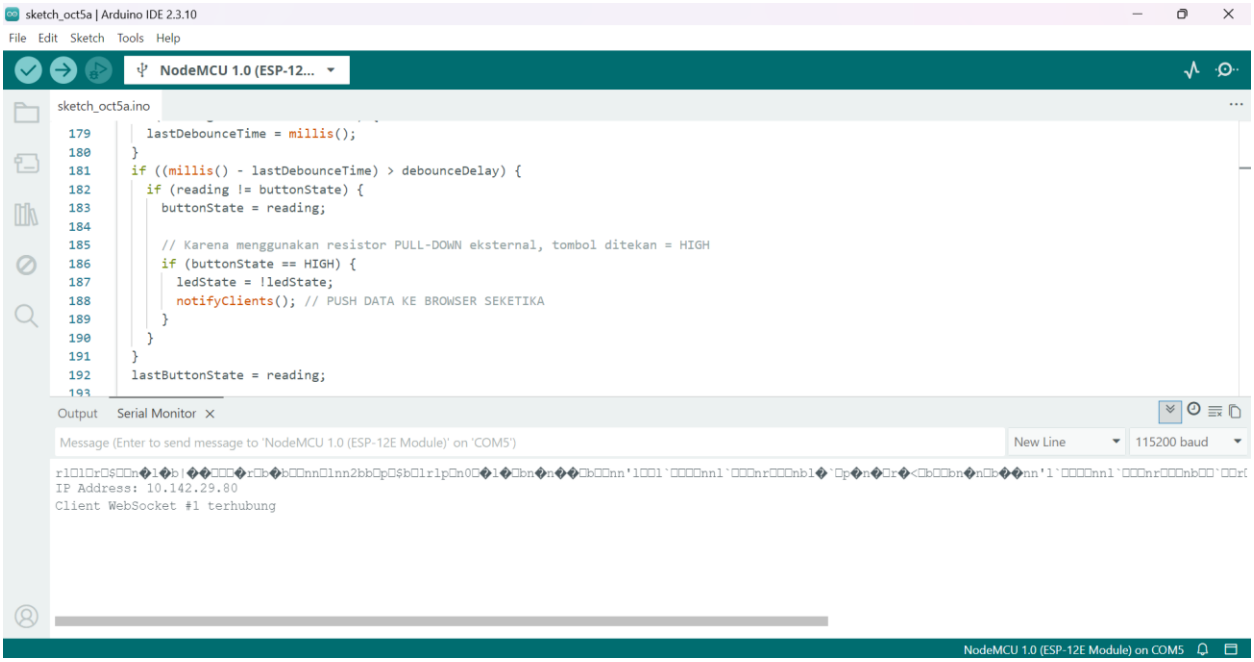
```
170  void loop() {
171    ws.cleanupClients();
172
173    // 1. Eksekusi perangkat keras LED
174    digitalWrite(ledPin, ledState ? HIGH : LOW);
175
176    // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
177    int reading = digitalRead(buttonPin);
178    if (reading != lastButtonState) {
179      lastDebounceTime = millis();
180    }
181    if ((millis() - lastDebounceTime) > debounceDelay) {
182      if (reading != buttonState) {
183        buttonState = reading;
184
185        // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
186        if (buttonState == HIGH) {
187          ledState = !ledState;
188          notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
189        }
190      }
191    }
192    lastButtonState = reading;
```

```
181    if ((millis() - lastDebounceTime) > debounceDelay) {
182      if (reading != buttonState) {
183        buttonState = reading;
184
185        // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
186        if (buttonState == HIGH) {
187          ledState = !ledState;
188          notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
189        }
190      }
191    }
192    lastButtonState = reading;
193
194    // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
195    if ((millis() - lastTime) > 3000) {
196      float t = dht.readTemperature();
197      if(!isnan(t)) {
198        currentTemp = String(t);
199        notifyClients(); // Push pembaruan suhu otomatis
200      }
201      lastTime = millis();
202    }
203  }
```

Rangkaian

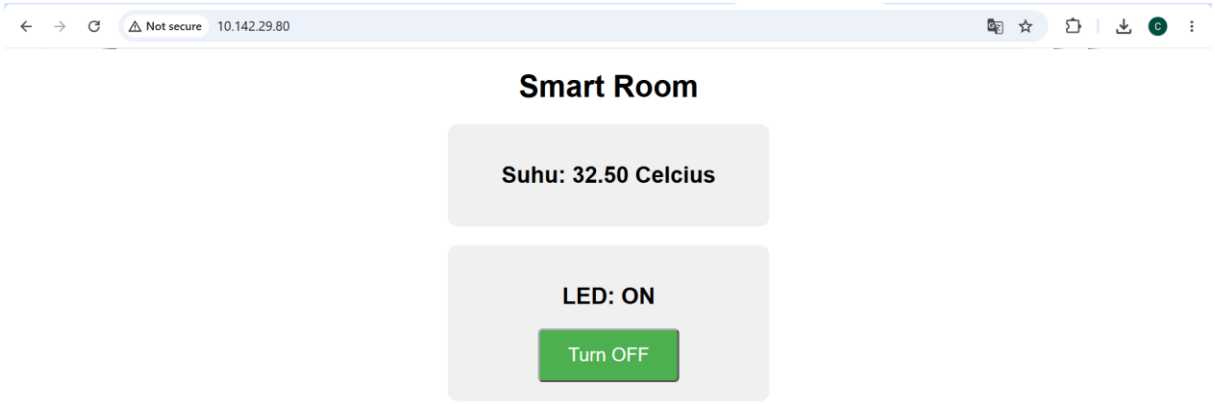


Mendapatkan IP

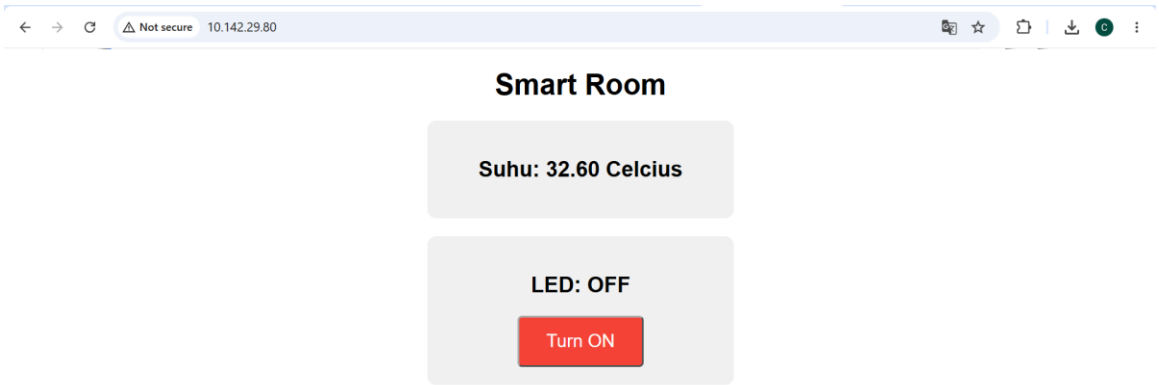


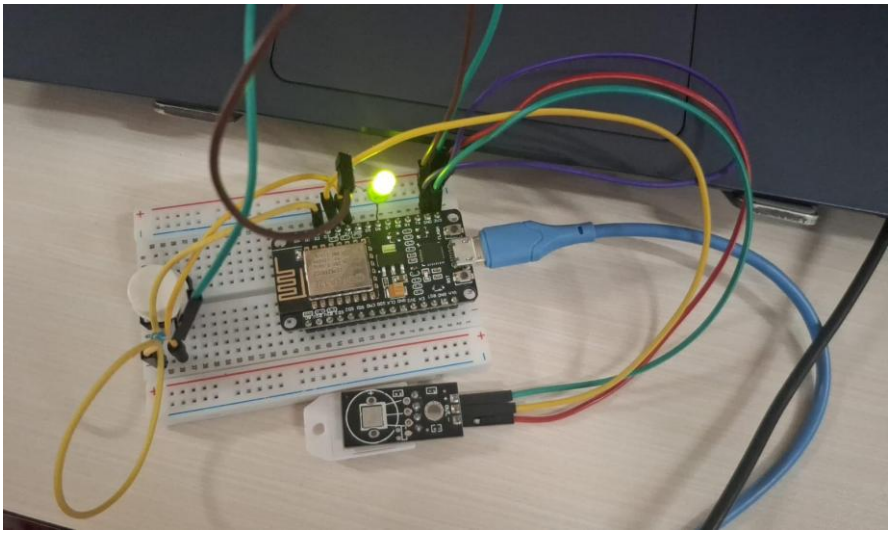
Tampilan web socket

Ketika pencet “Turn off” LED akan mati begitu sebaliknya jika button di pencet,



Ketika pencet “Turn on” LED akan hidup begitu sebalik nya jika button di pencet.





4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

  Jawab: WebSocket lebih hemat bandwidth karena koneksinya tetap terbuka, jadi data bisa langsung dikirim ketika ada perubahan tanpa harus melakukan refresh halaman terus-menerus.

  Sedangkan pada Meta Refresh HTTP, halaman harus dimuat ulang berkali-kali untuk mendapatkan data terbaru. Akibatnya, penggunaan data lebih banyak dan halaman bisa terlihat berkedip.

  Jadi, WebSocket lebih hemat, lebih cepat, dan cocok untuk menampilkan data secara real-time.
- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

  Jawab: Jika menggunakan delay(3000) di dalam void loop(), NodeMCU akan berhenti selama 3 detik dan tidak bisa langsung memproses komunikasi WebSocket.

  Akibatnya, respon dari tombol atau browser menjadi lambat, data WebSocket bisa terlambat dikirim, dan sistem tidak lagi bekerja secara *real-time*. Jadi, delay(3000) dapat mengganggu komunikasi antara browser dan NodeMCU.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

  Jawab:Nama ID elemen HTML yang menjadi target perubahan teks oleh JavaScript adalah:
  - ledStatus: untuk mengubah tulisan status LED menjadi ON atau OFF.
  - toggleBtn**:** untuk mengubah tulisan tombol menjadi Turn ON atau Turn OFF**.**

  Jadi, ID yang digunakan adalah ledStatus dan toggleBtn**.**
- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
  i. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
     - Tambahkan kode di bagian : “ String currentHum = "--";)”

```
16  // Variabel Pelacak Status (State & Cache)
17  bool ledState = false;
18  String currentTemp = "--";
19  String currentHum = "--";
20  int buttonState;
21  int lastButtonState = LOW;
22  unsigned long lastDebounceTime = 0;
23  const unsigned long debounceDelay = 50;
24  unsigned long lastTime = 0;
25
```

     - Tambahkan kode di bagian notifyClients() : jsonString += "\"hum\":\"" + currentHum + "\"}";

```
110 // Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
111 void notifyClients() {
112   String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
113   jsonString += "\"suhu\":\"" + currentTemp + "\"}";
114   jsonString += "\"hum\":\"" + currentHum + "\"}";
115   ws.textAll(jsonString);
116 }
```

  ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

- Tambahkan kode di bagian ini

```
// 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
if ((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  if(!isnan(t)) {
    currentTemp = String(t);
    notifyClients(); // Push pembaruan suhu otomatis
  }
  lastTime = millis();
}
```

- Tambahkan kode di bagian ini

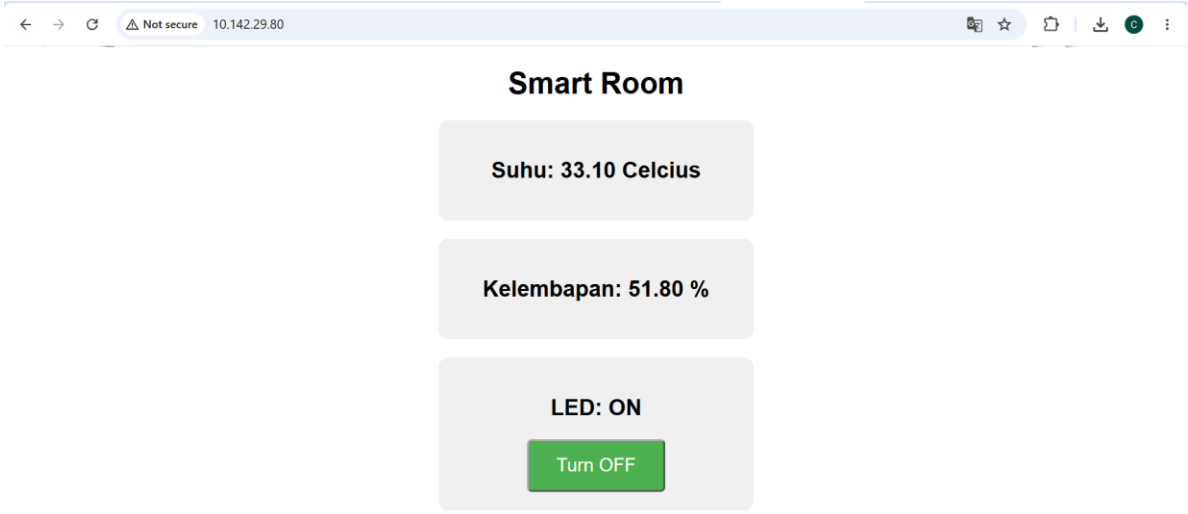
```
<h1>Smart Room</h1>
<div class="card">
  <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
</div>
<div class="card">
  <h2>LED: <span id="ledStatus">OFF</span></h2>
  <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
</div>
```

- Tambahkan kode di bagian ini

```
// Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
if(dataObj.suhu !== undefined) {
  document.getElementById('tempValue').innerHTML = dataObj.suhu;
}

if(dataObj.hum !== undefined) {
  document.getElementById('humValue').innerHTML = dataObj.hum;
}
```

- **Output dari dashboard web socket yang di tambahkan kelembapan**



5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

- **MAC ADDRESS YANG SAYA DAPAT:**



5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

- **Saya sebagai Penerima (Slave)**
  Kode:



```
  Serial.print("Perintah ID: ");
  Serial.println(paketTerima.perintahId);
  Serial.print("Nilai Parameter: ");
  Serial.println(paketTerima.nilaiParameter);
  Serial.println("-----------------------------------");
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW Gagal!");
    return;
  }

  // Tetapkan peran board sebagai Slave
  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
  esp_now_register_recv_cb(OnDataRecv);

  Serial.println("Receiver ESP-NOW Menunggu Paket...");
}

void loop() {
```

```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW Gagal!");
    return;
  }

  // Tetapkan peran board sebagai Slave
  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
  esp_now_register_recv_cb(OnDataRecv);

  Serial.println("Receiver ESP-NOW Menunggu Paket...");
}

void loop() {
  // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
}
```

- **Paket di terima dari si pengirim (sekar)**

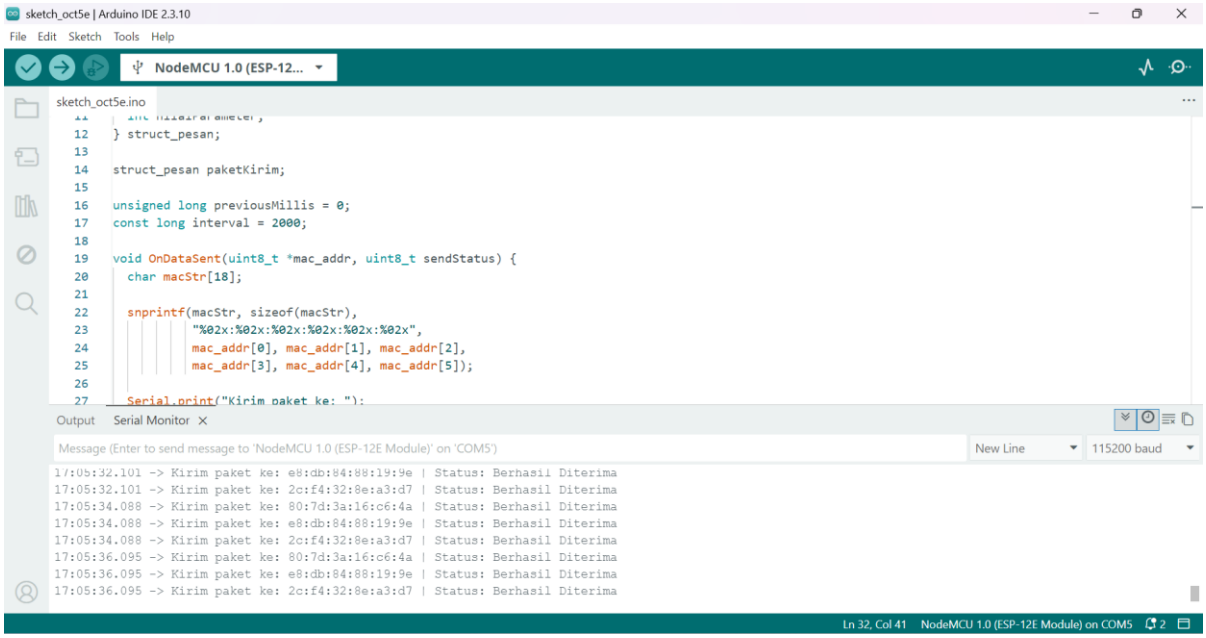


Sekarang saya jadi pengirim:

- **MAC ADDRESS PUNYA TEMAN TEMAN SAYA:**

```
4   // MAC Address Receiver
5   uint8_t receiver1[] = {0x80, 0x7D, 0x3A, 0x16, 0xC6, 0x4A}; // Sekar
6   uint8_t receiver2[] = {0xE8, 0xDB, 0x84, 0x88, 0x19, 0x9E}; // Lilis
7   uint8_t receiver3[] = {0x2C, 0xF4, 0x32, 0x8E, 0xA3, 0xD7}; // Mita
```

- **Berhasil dikirim dan diterima oleh penerima**



- **Kode**



5.5.2_pengirim.ino

```
  Serial.print("Kirim paket ke: ");
  Serial.print(macStr);
  Serial.print(" | Status: ");

  if (sendStatus == 0) {
    Serial.println("Berhasil Diterima");
  } else {
    Serial.println("Gagal (Tidak Terjangkau)");
  }
}

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != 0) {
    Serial.println("Gagal menginisialisasi ESP-NOW!");
    return;
  }

  esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
  esp_now_register_send_cb(OnDataSent);

  // Daftarkan Receiver Sela
  esp_now_add_peer(
    receiver1,
    receiver1,
    ESP_NOW_ROLE_SLAVE,
    1,
    NULL,
    0
  );

  // Daftarkan Receiver Lilis
  esp_now_add_peer(
    receiver2,
    ESP_NOW_ROLE_SLAVE,
    1,
    NULL,
    0
  );

  // Daftarkan Receiver Mita
  esp_now_add_peer(
    receiver3,
    ESP_NOW_ROLE_SLAVE,
    1,
    NULL,
    0
  );

  Serial.println("Controller Sekar ESP-NOW Siap!");
}
```

```
  esp_now_add_peer(
    receiver3,
    ESP_NOW_ROLE_SLAVE,
    1,
    NULL,
    0
  );

  Serial.println("Controller Sekar ESP-NOW Siap!");
}

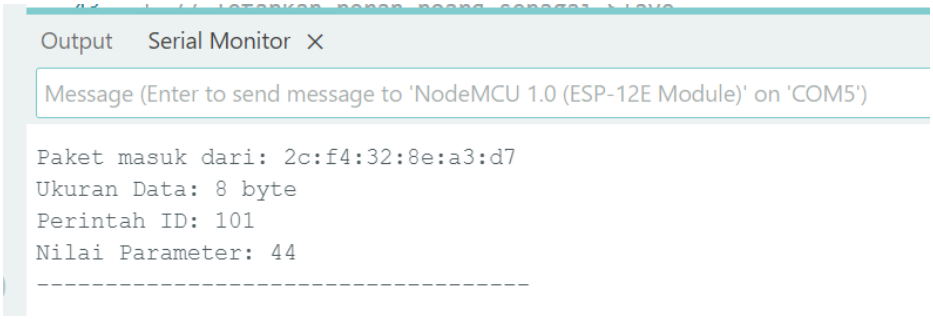
void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

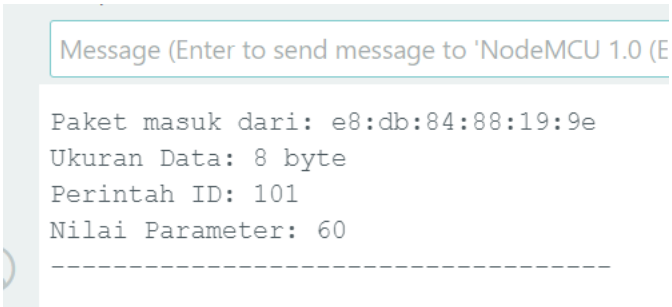
    paketKirim.perintahId = 101;
    paketKirim.nilaiParameter = random(10, 100);

    // 0 = kirim ke semua peer yang sudah didaftarkan
    esp_now_send(
      0,
      (uint8_t *) &paketKirim,
      sizeof(paketKirim)
    );
  }
}
```

- **Paket di terima dari si pengirim (mita)**



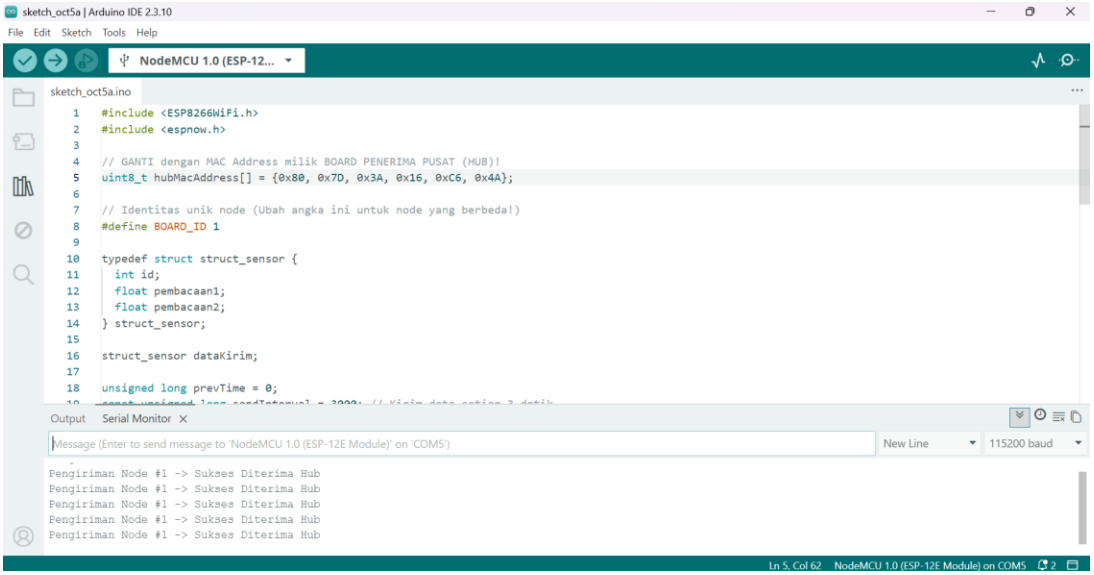
- **Paket di terima dari si pengirim (lilis)**



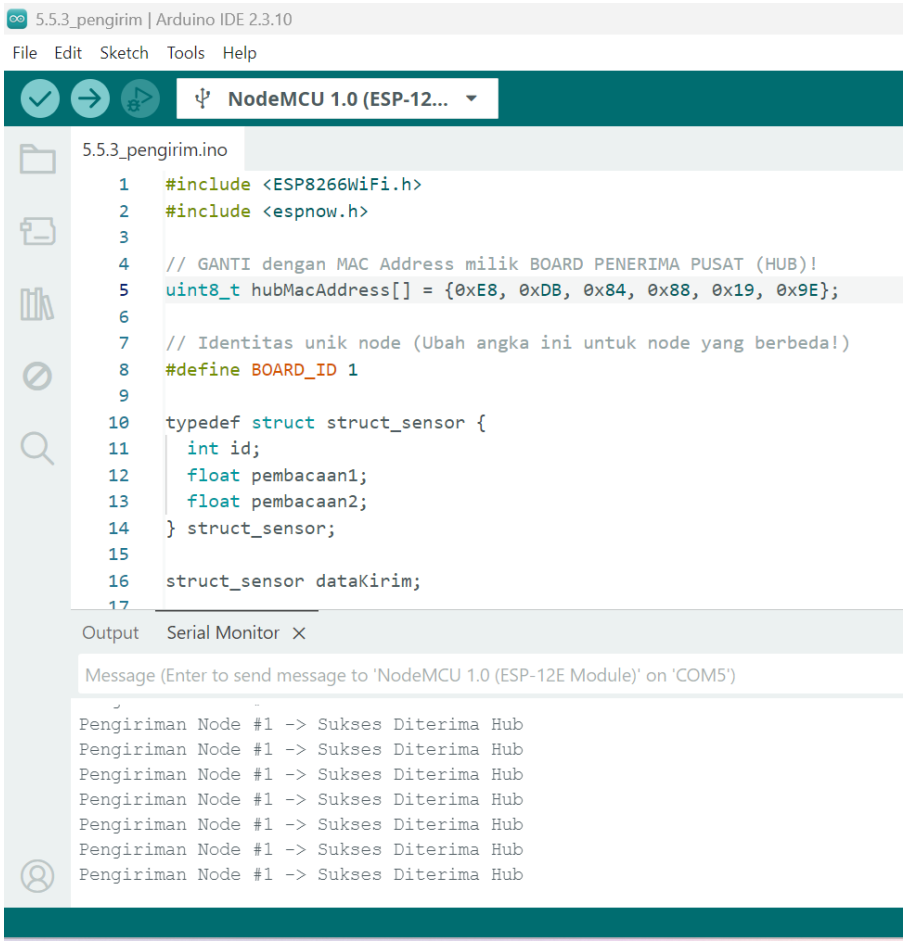
5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*)

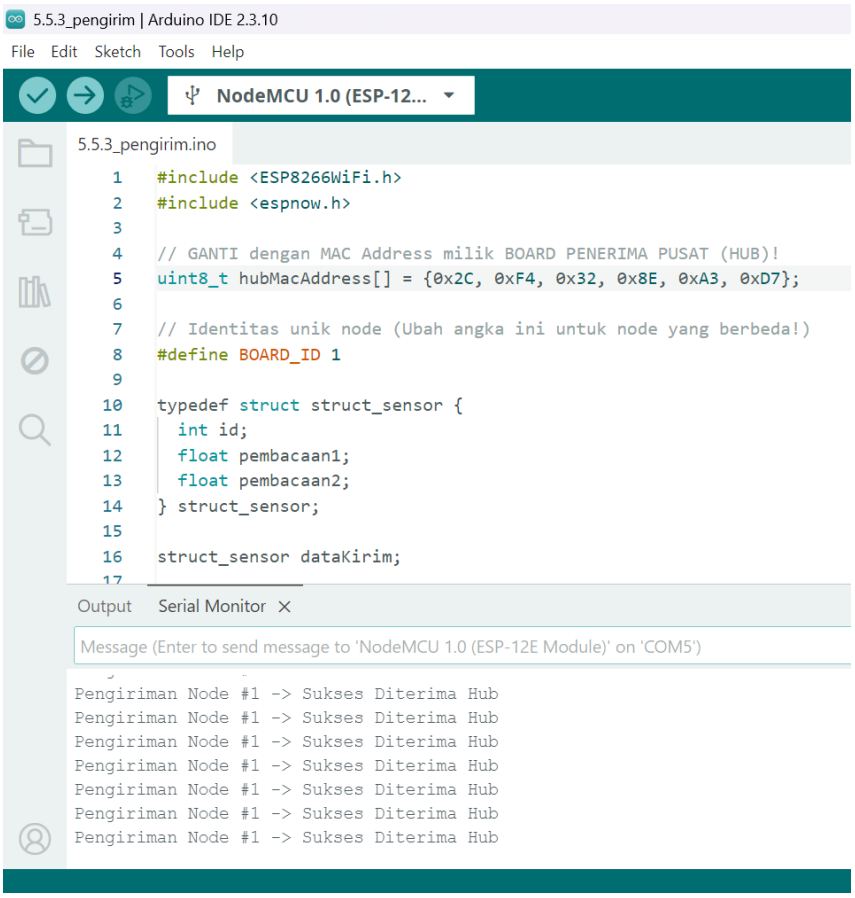
- **Saya sebagai Pengirim ke server dan di terima oleh Hub (sekar)**



- **Saya sebagai Pengirim ke server dan di terima oleh Hub (lilis)**



- **Saya sebagai Pengirim ke server dan di terima oleh Hub (mita)**



- **Kode yang di gunakan:**



```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW gagal");
    return;
  }

  esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
  esp_now_register_send_cb(OnDataSent);

  // Daftarkan hub penerima tunggal
  esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
  unsigned long now = millis();
  if (now - prevTime >= sendInterval) {
    prevTime = now;

    dataKirim.id = BOARD_ID;
    dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
    dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %
```

```
  esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
  esp_now_register_send_cb(OnDataSent);

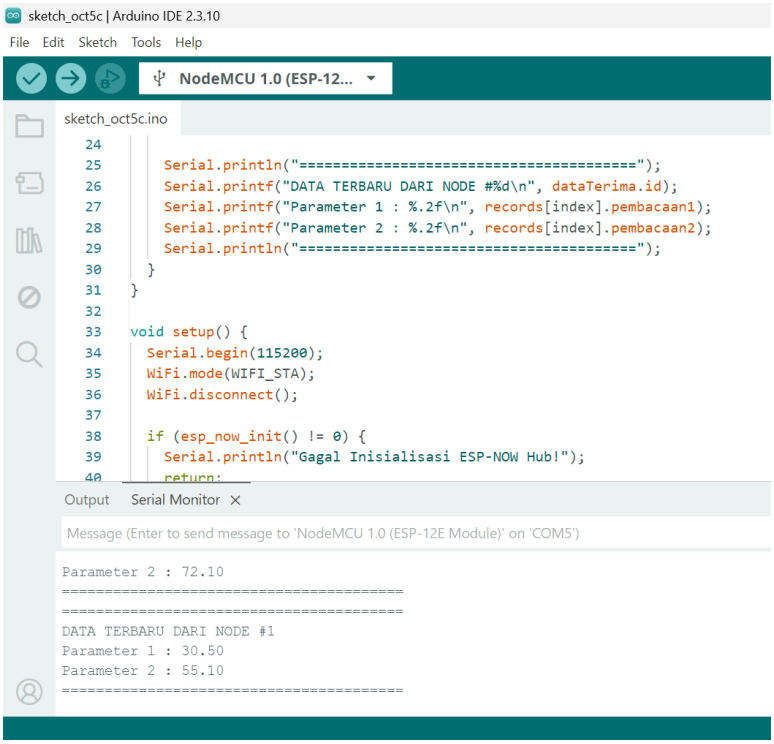
  // Daftarkan hub penerima tunggal
  esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
  unsigned long now = millis();
  if (now - prevTime >= sendInterval) {
    prevTime = now;

    dataKirim.id = BOARD_ID;
    dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
    dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %

    esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
  }
}
```

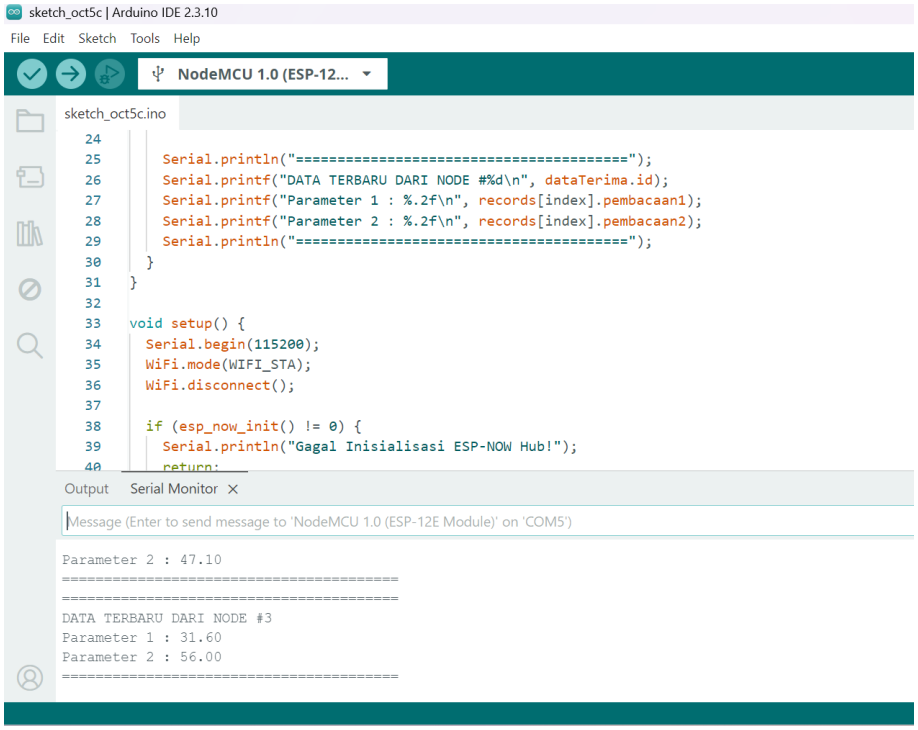
- **Saya sebagai penerima server node**
  - **Node 1**



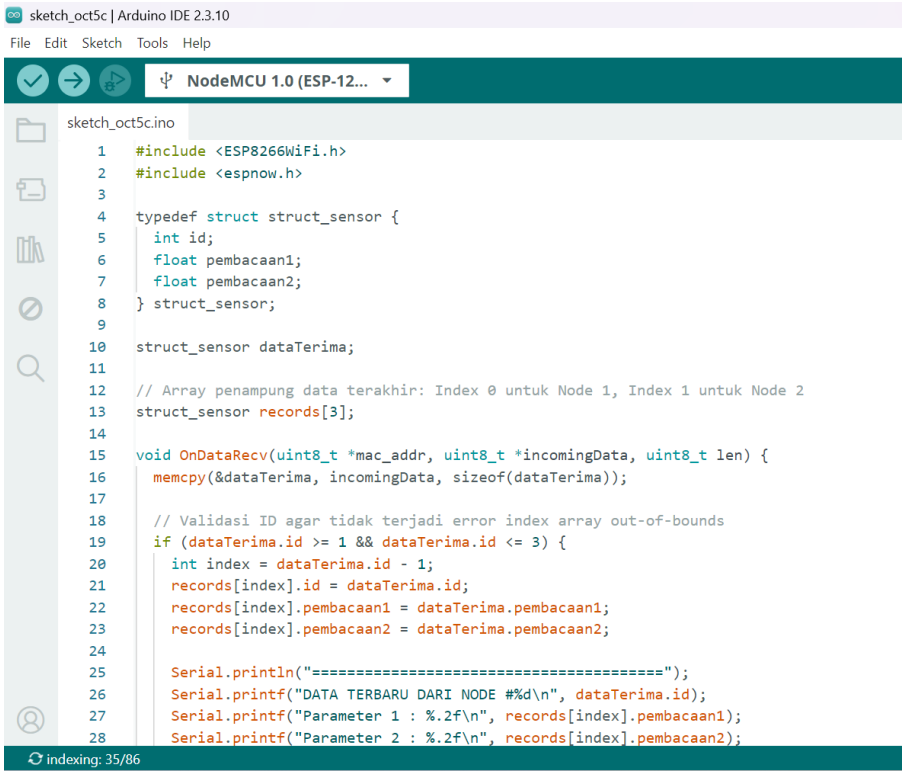
  - **Node 2**



- **Node 3**



- **Kode yang di gunakan:**

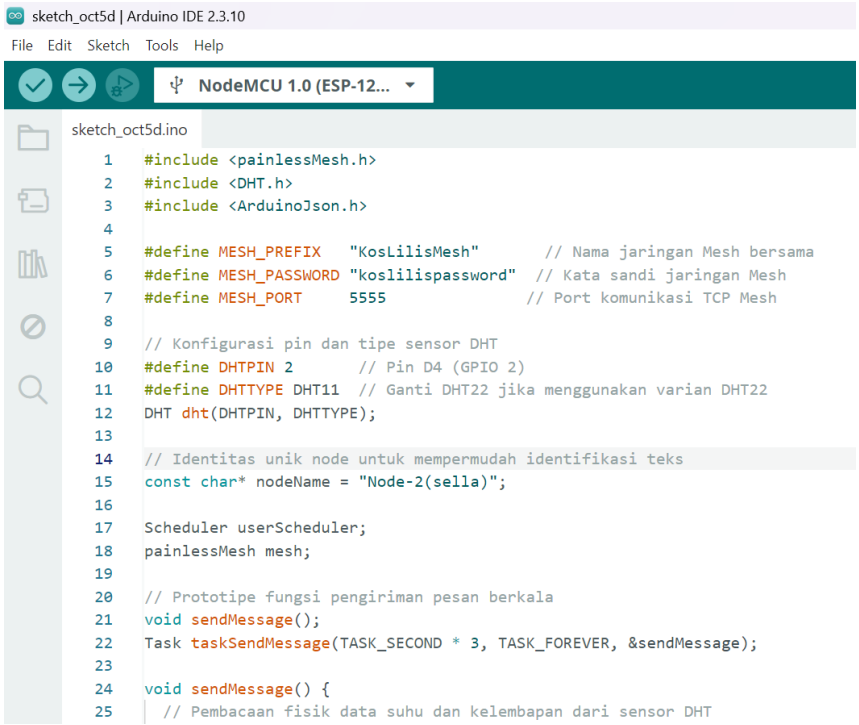


```
28      Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
29      Serial.println("========================================");
30    }
31  }
32
33  void setup() {
34    Serial.begin(115200);
35    WiFi.mode(WIFI_STA);
36    WiFi.disconnect();
37
38    if (esp_now_init() != 0) {
39      Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
40      return;
41    }
42
43    esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
44    esp_now_register_recv_cb(OnDataRecv);
45
46    Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
47  }
48
49  void loop() {
50    // Loop bebas menangani tugas lain
51  }
```

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

- **Kode yang digunakan:**




```
1   #include <painlessMesh.h>
2   #include <DHT.h>
3   #include <ArduinoJson.h>
4
5   #define MESH_PREFIX   "KosLilisMesh"       // Nama jaringan Mesh bersama
6   #define MESH_PASSWORD "koslilispassword"  // Kata sandi jaringan Mesh
7   #define MESH_PORT     5555               // Port komunikasi TCP Mesh
8
9   // Konfigurasi pin dan tipe sensor DHT
10  #define DHTPIN 2       // Pin D4 (GPIO 2)
11  #define DHTTYPE DHT11  // Ganti DHT22 jika menggunakan varian DHT22
12  DHT dht(DHTPIN, DHTTYPE);
13
14  // Identitas unik node untuk mempermudah identifikasi teks
15  const char* nodeName = "Node-2(sella)";
16
17  Scheduler userScheduler;
18  painlessMesh mesh;
19
20  // Prototipe fungsi pengiriman pesan berkala
21  void sendMessage();
22  Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24  void sendMessage() {
25    // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
```

```
  float suhu = dht.readTemperature();
  float kelembapan = dht.readHumidity();

  // Validasi pembacaan sensor
  if (isnan(suhu) || isnan(kelembapan)) {
    Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
    return;
  }

  // Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
  StaticJsonDocument<200> doc;
  doc["node"] = nodeName;
  doc["chipId"] = mesh.getNodeId();
  doc["suhu"] = suhu;
  doc["kelembapan"] = kelembapan;

  String msg;
  serializeJson(doc, msg);

  // Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
  mesh.sendBroadcast(msg);

  Serial.print("[KIRIM MESH] ");
  Serial.println(msg);
}

// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
void receivedCallback(uint32_t from, String &msg) {
  // Parsing muatan teks JSON yang diterima
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, msg);

  if (!error) {
    const char* sender = doc["node"];
    uint32_t chipId   = doc["chipId"];
    float suhu        = doc["suhu"];
    float kelembapan  = doc["kelembapan"];

    Serial.println("========================================");
    Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
    Serial.printf("Suhu       : %.2f °C\n", suhu);
    Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
    Serial.println("========================================");
  } else {
    Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
  }
}

// Callback otomatis saat ada perangkat baru bergabung ke mesh
void newConnectionCallback(uint32_t nodeId) {
  Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
```

```
// Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
void changedConnectionCallback() {
  Serial.println("--> Topologi rantai mesh telah diperbarui");
}

void setup() {
  Serial.begin(115200);

  // Inisialisasi sensor DHT
  dht.begin();

  // Atur level log debugging (hanya tampilkan error dan status startup)
  mesh.setDebugMsgTypes(ERROR | STARTUP);

  // Inisialisasi jaringan mesh
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

  // Daftarkan event callback
  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);
  mesh.onChangedConnections(&changedConnectionCallback);

  // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();
```

```
  // Atur level log debugging (hanya tampilkan error dan status startup)
  mesh.setDebugMsgTypes(ERROR | STARTUP);

  // Inisialisasi jaringan mesh
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

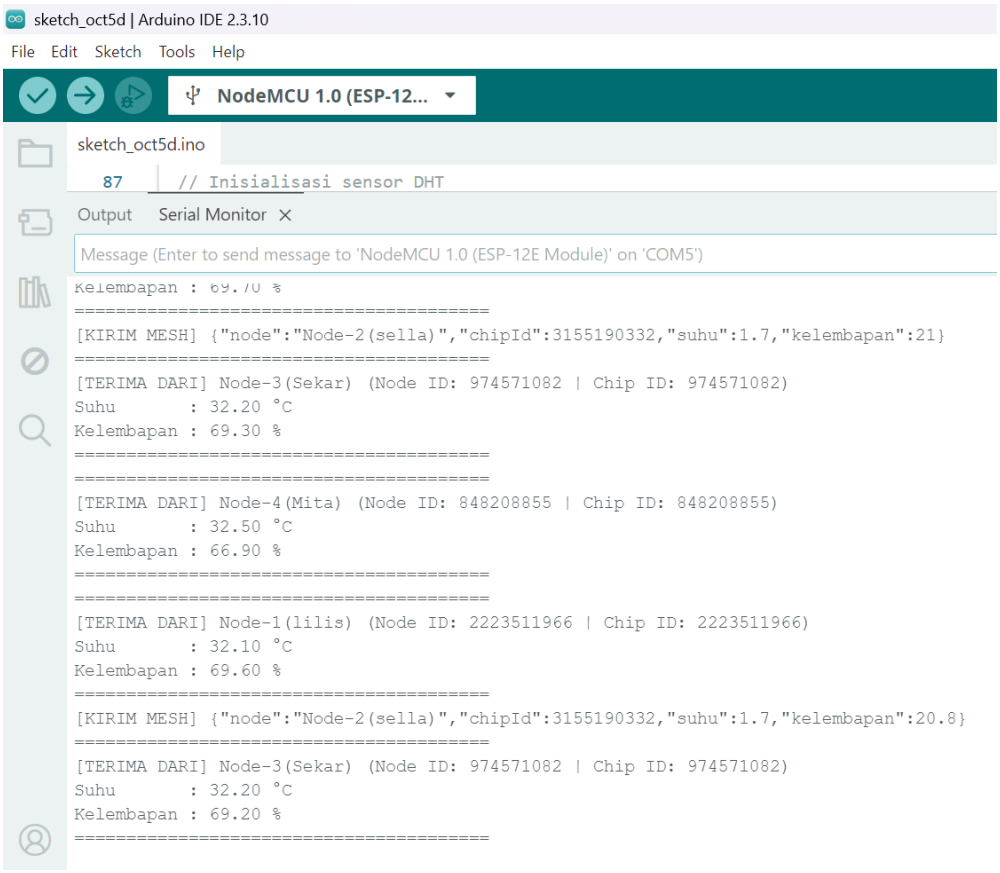
  // Daftarkan event callback
  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);
  mesh.onChangedConnections(&changedConnectionCallback);

  // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();

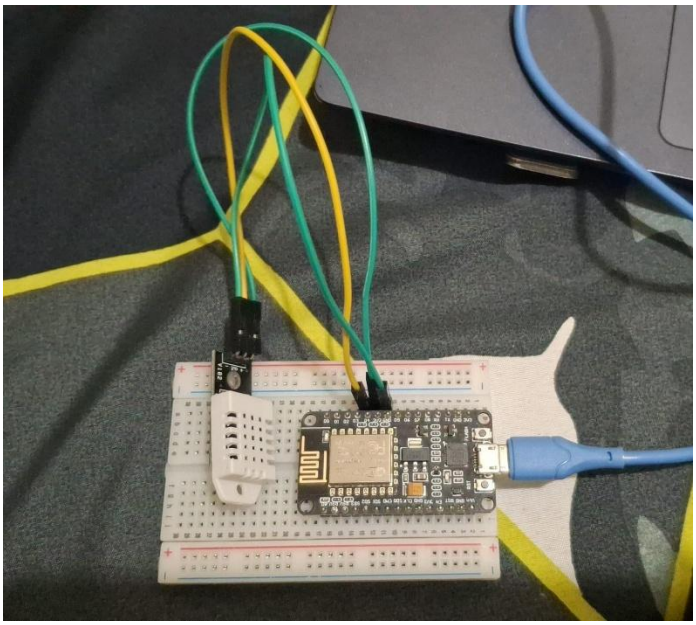
  Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
}

void loop() {
  // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
  mesh.update();
}
```

- **Output di Serial Monitor:**



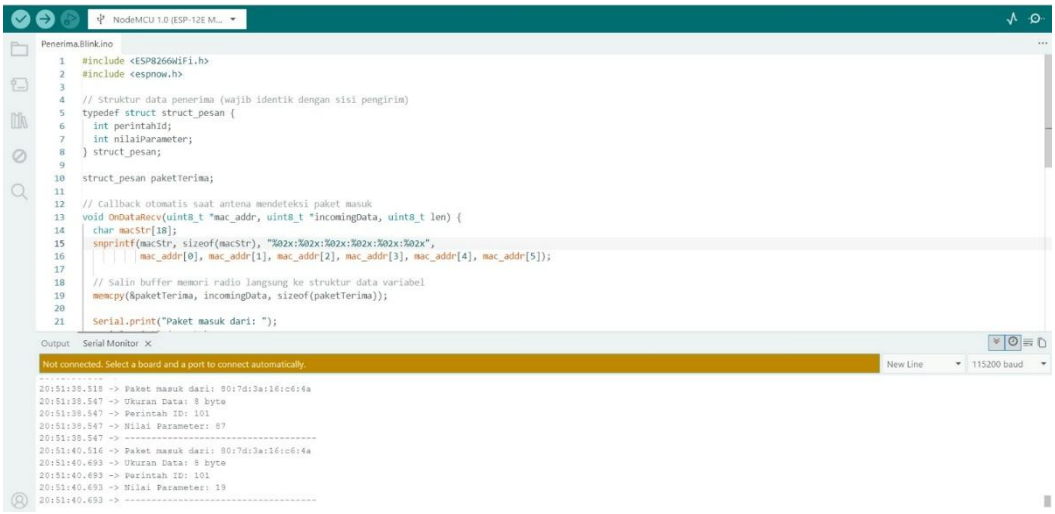
- **Rangkaian yang digunakan:**



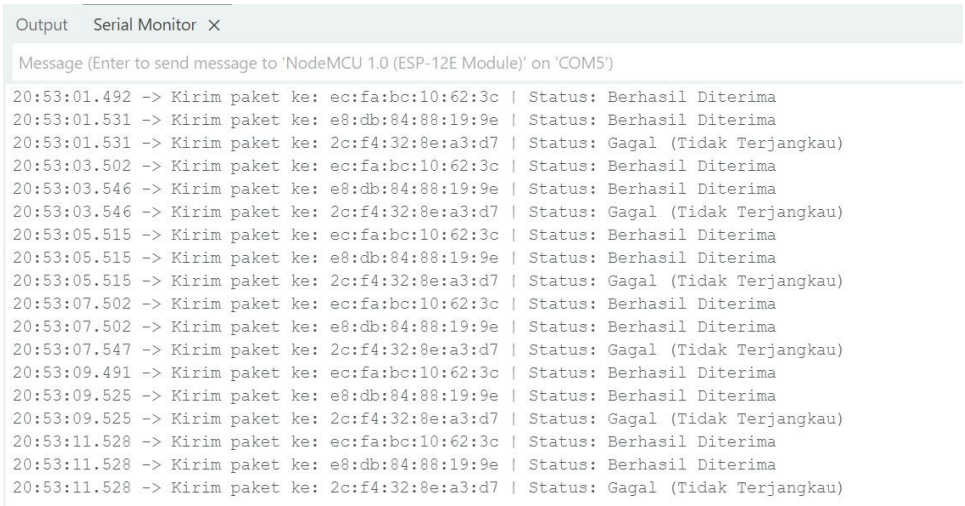
5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0), lakukan eksperimen pemutusan simpul:
  i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
  ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- **Node 2 dicabut kabel USB nya:**



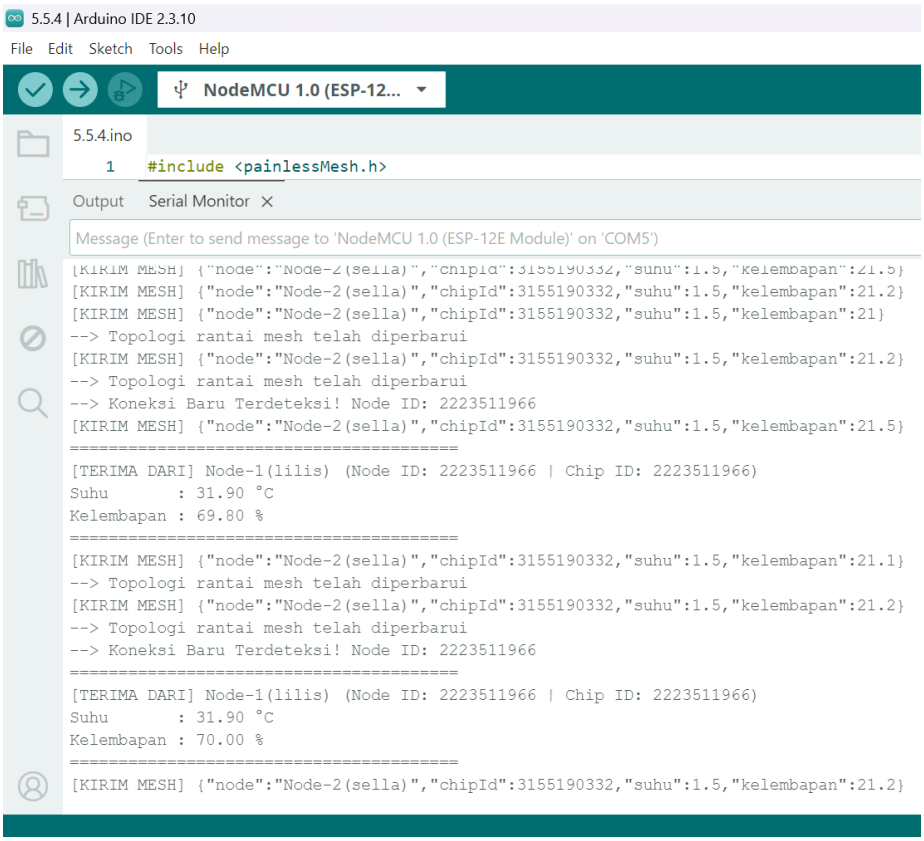
- **Hasil nya: Gagal (Tidak Terjangkau)**



iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawab: ESP-NOW dapat mendeteksi kegagalan karena komunikasi dilakukan langsung menggunakan alamat MAC perangkat. Jika perangkat tujuan mati atau tidak terjangkau, paket yang dikirim tidak mendapat respons dari perangkat tersebut, sehingga ESP-NOW memberikan status "Gagal (Tidak Terjangkau)". Jadi, ESP-NOW bisa mengetahui pengiriman berhasil atau gagal tanpa harus menggunakan TCP.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID: ....
  i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* changedConnectionCallback() untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
  ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!
- **Output yang di hasilkan:**



- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
    i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
    ii. Jika suhu bernilai normal (≤32.0 ∘C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.
    iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda!
- **Blok kode yang di modifikasi:**

latihan_3.ino

```
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "KosLilisMesh"       // Nama jaringan Mesh bersama
#define MESH_PASSWORD "koslilispassword"  // Kata sandi jaringan Mesh
#define MESH_PORT     5555               // Port komunikasi TCP Mesh

// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2       // Pin D4 (GPIO 2)
#define DHTTYPE DHT22  // Ganti DHT22 jika menggunakan varian DHT22
DHT dht(DHTPIN, DHTTYPE);

// Identitas unik node
const char* nodeName = "Node-3(Sekar)";

Scheduler userScheduler;
painlessMesh mesh;

// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void sendMessage() {
  // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
  float suhu = dht.readTemperature();
  float kelembapan = dht.readHumidity();
```

latihan_3.ino

```
28
29    // Validasi pembacaan sensor
30    if (isnan(suhu) || isnan(kelembapan)) {
31      Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
32      return;
33    }
34
35    // Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
36    StaticJsonDocument<200> doc;
37    doc["node"] = nodeName;
38    doc["chipId"] = mesh.getNodeId();
39    doc["suhu"] = suhu;
40    doc["kelembapan"] = kelembapan;
41
42    String msg;
43    serializeJson(doc, msg);
44
45    // Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
46    mesh.sendBroadcast(msg);
47
48    Serial.print("[KIRIM MESH] ");
49    Serial.println(msg);
50  }
51
52  // Callback otomatis saat menerima pesan dari node manapun di jaringan mesh (Latihan 3 Filter Telemetri)
53  void receivedCallback(uint32_t from, String &msg) {
54    // Parsing muatan teks JSON yang diterima
55    StaticJsonDocument<200> doc;
56    DeserializationError error = deserializeJson(doc, msg);
57
58    if (!error) {
59      const char* sender = doc["node"];
60      uint32_t chipId    = doc["chipId"];
61      float suhu         = doc["suhu"];
62      float kelembapan   = doc["kelembapan"];
63
64      // Filter Ambang Batas Suhu Kritis (> 32.0 °C)
65      if (suhu > 32.0) {
66        Serial.println("========================================");
67        Serial.println("        [PERINGATAN SUHU KRITIS]        ");
68        Serial.println("========================================");
69        Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
70        Serial.printf("Suhu       : %.2f °C\n", suhu);
71        Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
72        Serial.println("========================================");
73      } else {
74        // Format ringkas jika suhu normal (<= 32.0 °C)
75        Serial.printf("[NORMAL] Telemetri dari Node %u aman.\n", chipId);
76      }
77    } else {
78      Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
79    }
80  }
81
82  // Callback otomatis saat ada perangkat baru bergabung ke mesh
83  void newConnectionCallback(uint32_t nodeId) {
84    Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
85  }
```

latihan_3.ino

```
86
87  // Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
88  void changedConnectionCallback() {
89    Serial.println("--> Topologi rantai mesh telah diperbarui");
90  }
91
92  void setup() {
93    Serial.begin(115200);
94
95    // Inisialisasi sensor DHT
96    dht.begin();
97
98    // Atur level log debugging (hanya tampilkan error dan status startup)
99    mesh.setDebugMsgTypes(ERROR | STARTUP);
100
101   // Inisialisasi jaringan mesh
102   mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
103
104   // Daftarkan event callback
105   mesh.onReceive(&receivedCallback);
106   mesh.onNewConnection(&newConnectionCallback);
107   mesh.onChangedConnections(&changedConnectionCallback);
108
109   // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
110   userScheduler.addTask(taskSendMessage);
111   taskSendMessage.enable();
112
113   Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
114 }
115
116 void loop() {
117   // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
118   mesh.update();
119 }
```

- **Output yang di hasilkan:**

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```
21:18:30.431 -> [NORMAL] Telemetri dari Node 3155190332 aman.
21:18:30.632 -> [NORMAL] Telemetri dari Node 2223511966 aman.
21:18:31.375 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.2,"kelembapan":69.5}
21:18:32.423 -> [NORMAL] Telemetri dari Node 848208855 aman.
21:18:33.416 -> [NORMAL] Telemetri dari Node 3155190332 aman.
21:18:33.617 -> [NORMAL] Telemetri dari Node 2223511966 aman.
21:18:34.350 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.2,"kelembapan":69.2}
21:18:35.398 -> [NORMAL] Telemetri dari Node 848208855 aman.
21:18:36.500 -> [NORMAL] Telemetri dari Node 3155190332 aman.
21:18:36.765 -> [NORMAL] Telemetri dari Node 2223511966 aman.
21:18:37.350 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.2,"kelembapan":69}
21:18:38.375 -> [NORMAL] Telemetri dari Node 848208855 aman.
21:18:39.429 -> [NORMAL] Telemetri dari Node 3155190332 aman.
21:18:39.583 -> [NORMAL] Telemetri dari Node 2223511966 aman.
21:18:40.375 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.2,"kelembapan":69}
21:18:41.415 -> [NORMAL] Telemetri dari Node 848208855 aman.
21:18:42.416 -> [NORMAL] Telemetri dari Node 3155190332 aman.
21:18:42.531 -> [NORMAL] Telemetri dari Node 2223511966 aman.
```